Project Specification: University Resource Booking System

1. Project Overview

The objective of this project is to develop a command-line interface (CLI) application using **TypeScript** and **Modern JavaScript (ES6+)**. The system manages university resources (such as Labs, Lecture Halls, and Projectors) and handles booking requests made by students or professors, ensuring strict type-safety and structural integrity.

2. Domain Models & Data Structures

You are required to define the following data structures using TypeScript **Enums**, **Interfaces**, and **Types**:

A. Enums

- ResourceType: Valid values are LAB, LECTURE_HALL, PROJECTOR.
- BookingStatus: Valid values are PENDING, APPROVED, CANCELLED.

B. Interfaces

- Resource:
  - id (number)
  - name (string)
  - type (ResourceType)
  - isAvailable (boolean)
- Booking:
  - bookingId (number)
  - resourceId (number)
  - userEmail (string)
  - durationInHours (number)
  - status (BookingStatus)
  - createdAt (Date)

C. Advanced Types & Utility Types

- NewBookingInput: Represents the data required to create a booking. It must omit the system-generated fields. Use **Omit<Booking, 'bookingId' 'createdAt' 'status' |>**.
- UpdateBookingInput: Represents the data needed to update an existing booking. All fields should be optional. Use **Partial<Booking>**.

3. Core Functional Requirements (Class Design)

You must implement a generic management class **EntityManager<T>** that handles core CRUD operations using an in-memory array.

A. Generic Manager: EntityManager<T>

- private data: T[] = []
- add(item: T): void: Appends an item to the collection.
- getAll(): T[]: Returns all items.
- getById(idKey: keyof T, idValue: any): T | undefined: Generic search method.

B. System Operations (Business Logic)

Create specialized functions (or a separate service class) using **Arrow Functions** to implement the following features:

1. **createBooking(input: NewBookingInput): Booking**:
    - Generates a random/incremental bookingId.
    - Sets the initial status to PENDING.
    - Sets createdAt to the current date.
    - Uses the **Spread Operator (...)** to merge system fields with user input.
2. **updateBookingStatus(id: number, update: UpdateBookingInput): Booking**:
    - Finds the booking and updates its details using modern array mapping.
3. **calculateTotalBookingHours(bookings: Booking[]): number**:
    - Utilizes **Array.prototype.reduce()** to calculate the cumulative hours of all approved bookings.
4. **filterResourcesByType(resources: Resource[], type: ResourceType): Resource[]**:
    - Utilizes **Array.prototype.filter()** to return matching resources.
5. **generateBookingSummary(bookings: Booking[], resources: Resource[]): any[]**:
    - Utilizes **Array.prototype.map()** and **Destructuring** to return a custom report object containing the booking ID, user email, and the specific resource name.

4. Execution Flow (Simulation script index.ts)

Your script must execute the following sequential instructions to demonstrate system functionality:

1. **Initialization**: Instantiate an EntityManager<Resource> and seed it with at least 3 resources (e.g., "Main AI Lab", "Hall 404").
2. **Creation**: Invoke createBooking twice using the NewBookingInput structure to book different resources.
3. **Transformation**: Update one booking status to APPROVED and another to CANCELLED via your update functions.
4. **Reporting**: Print the outputs of calculateTotalBookingHours and generateBookingSummary to the console using **Template Strings**.

5. Deliverables & Evaluation Criteria

- **Type Safety**: No usage of the any type (except where specified in the generic manager).
- **Compilation**: The project must compile successfully without errors using tsc.
- **Code Style**: Proper encapsulation, explicit function return types, and clean variable scoping (let/const).